FAQ-22: What Happened to My PurePath Timings in Grail?

> **Series:** FAQ — Frequently Asked Questions | **Reference:** 22 — PurePath Timings in the Grail Span Model | **Created:** August 2026 | **Last Updated:** 10/15/2026

Overview

Teams arriving from a classic Dynatrace estate — or from another APM tool — tend to ask a version of the same question within the first week of running `fetch spans`:

> *"Where did IO time, lock time, and fetch count go? Why does an outgoing HTTP call have no service and no endpoint? And can I rely on `request_attribute.<name>`?"*

All three have the same root. The Grail span model is **not** a serialization of the classic PurePath detail view. It is a smaller, deliberately typed set of fields published as a **semantic dictionary** — and that dictionary is itself queryable, with a stability level on every field. Some classic sub-timings survived the move to Grail; most were never modeled. The ones that were not are not "lost in migration" and are not pending — they are simply not in the model.

This entry answers the three field-model questions directly, and then shows how to answer the next one without asking anyone. Two adjacent questions are handled by series that own them properly and are not restated here:

Adjacent question	Read
How to detect failures on client spans in practice	**SPANS-03** (troubleshooting) — `span.kind == "client" and span.status_code == "error"`
DPS trace Ingest / Retain / Query and who consumes query budget	**FINOPS-01** §6 and §9

Table of Contents

1. Short Answer
2. The Span Model Is a Queryable Contract
3. Which PurePath Timings Survived
4. Why an External Call Has No Service and No Endpoint
5. request_attribute vs captured_attribute
6. Answering the Next One Yourself
7. Recommended Approach
8. Summary and Next Steps

Prerequisites

Requirement	Details
Applies to	Any Grail tenant querying `fetch spans` — especially estates migrating from classic Dynatrace or from another APM tool
Audience	Anyone porting classic PurePath analysis, dashboards, or alerting onto span DQL
Permissions	`storage:buckets:read` plus `storage:spans:read` for the span queries — table permissions alone do not grant reads. The `dt.semantic_dictionary.*` queries scanned **zero bytes** on the validation tenant
Related topic series	SPANS (span DQL, troubleshooting, topology) · OTEL (OTLP ingest and attribute mapping) · FINOPS (trace Ingest / Retain / Query billing) · OPIPE (span processing at ingest)
Related FAQs	**FAQ-11** (how metrics work — the Classic/Grail dual-write that explains the metric half of this question) · **FAQ-16** (migrating classic entity selectors to Smartscape) · **FAQ-12** (coverage gaps from partial enablement)

> **Validation status.** Every field name, stability level, and description quoted in this entry was read from `dt.semantic_dictionary.fields` and `dt.semantic_dictionary.models` on a live SaaS tenant on **08/04/2026**, and the

> span-shape claims in sections 3–4 were confirmed against live `fetch spans` results on the same tenant. The one claim this entry does **not** make from observation is which platform apps consume trace Query budget — see section 7.

1. Short Answer

Three answers, each verifiable in your own tenant in about a minute.

- **CPU time survived. IO time, lock time, disk IO time, and suspension did not.** The Dynatrace span model publishes exactly two timing-breakdown fields — `span.timing.cpu` and `span.timing.cpu_self`, both `stable`. A search of the *entire* semantic dictionary for lock, wait, suspension, or IO timing fields returns nothing on spans. This is a model boundary, not a rollout gap, so treat it as permanent until the dictionary says otherwise — and the dictionary is how you would find out.
- **Mostly yes — an external call made while serving a request is a client span with no endpoint.** `endpoint.name` is `stable` and, per its own definition, *"exclusively detected on request root spans."* A client span beneath a server span is not a request root span, so `endpoint.name` is null on it and the `dt.service.request.*` metric family — which is dimensioned by service and endpoint — has no series for that call. The exception is a client span that *starts* a request — an outbound call from a scheduler, background thread or batch job with no inbound span above it. That span is the request root, and it carries an endpoint and a service-metric series (3.8% of client spans over 2 h on the validation tenant, 10/15/2026). Either way, you characterize outbound dependencies from the client spans themselves, grouped on `server.address` (`stable`). `peer.service` is not in the semantic dictionary — it appears only where OpenTelemetry instrumentation sets it, so treat it as optional.
- **`request_attribute.<name>` is `stable` — with two traps.** It is a registered field in the span model as `request_attribute.__attribute_name__`. But its type is **array**, not scalar, and it is *request-scoped reconciled* values. Its lesser-known sibling `captured_attribute.<name>` (also `stable`) holds the *span-scoped raw* values. Picking the wrong one, or comparing an array with `==`, are the two ways this goes wrong.

> **Sources:** Semantic Dictionary (DT docs), Request attributes (DT docs), Enhanced endpoints for SDv1 (DT docs). **Dictionary:** `endpoint.name` (`stable`), `request.is_failed` (`deprecated`), `span.timing.cpu` (`stable`), `span.timing.cpu_self` (`stable`),

> `server.address` (`stable`), `request.is_root_span` (`deprecated`, read 10/15/2026); quoted definitions and stability levels read 08/04/2026, re-read 08/27/2026 and 09/28/2026 unchanged. No row for `peer.service`, read 09/28/2026 (control: the same query returns `server.address`).

2. The Span Model Is a Queryable Contract

The reason this entry can answer "is it a permanent gap?" at all is that Dynatrace publishes the span model as data. Two tables carry it, both queryable with ordinary DQL and both scanning **zero billable bytes**:

Table	Grain	Use it for
`dt.semantic_dictionary.models`	One row per semantic model (`span`, `db`, `faas`, `ctg`, …)	"What is the complete field list for spans?"
`dt.semantic_dictionary.fields`	One row per field	"Does this field exist, what type is it, and is it stable?"

The `fields` table carries a **`stability`** column, and it is the single most useful thing in this entry. Its values separate three very different situations that all look identical in a query result:

`stability`	**What it means for you**
`stable`	Safe to build dashboards, alerts, and automation on
`experimental`	Real and populated, but the contract may change — do not build alerting on it
`deprecated`	Still present, on its way out — migrate off it

A field that is absent from `dt.semantic_dictionary.fields` entirely is a fourth case, and it is the one that matters for the PurePath timings: not experimental, not deprecated — **not modeled**.

> **Sources:** Semantic Dictionary (DT docs). **Dictionary:** the `stable` / `experimental` / `deprecated` ladder and the zero-bytes-scanned property observed directly, read 08/04/2026, re-read 08/27/2026 unchanged.

The whole span field list, in one query. Run this first — it is the ground truth for every "does field X exist?" argument:

```
// The complete published field list for Dynatrace spans.
// Scans zero billable bytes — this is metadata, not span data.
fetch dt.semantic_dictionary.models
| filter data_object == "spans"
| filter name == "span"
| fields title, description, fields
```

3. Which PurePath Timings Survived

The classic PurePath detail view breaks a request's elapsed time into contributions — CPU, IO, lock/sync, suspension, and so on. The Grail span model publishes **two** of them.

Classic PurePath concept	Grail span field	Stability	Status
CPU time (inclusive of same-thread children)	`span.timing.cpu`	`stable`	**Survived**
CPU time (this span only)	`span.timing.cpu_self`	`stable`	**Survived** — no classic equivalent as a first-class value
IO time	—	—	**Not modeled**
Disk IO time	—	—	**Not modeled**
Lock / sync / wait time	—	—	**Not modeled**

Classic PurePath concept	Grail span field	Stability	Status
Suspension time	—	—	**Not modeled**
Fetch count (rows fetched)	`db.result.fetch_size` — *nearest analogue only*	`experimental`	**Different measure** — see below

Two of those rows deserve care.

`span.timing.cpu` vs `span.timing.cpu_self`. The dictionary defines the first as *"the overall CPU time spent executing the span, including the CPU times of child spans that are running on the same thread on the same call stack,"* and the second as *"the CPU time spent exclusively on executing this span, not including the CPU times of any children."* The `_self` form is what you want for "which span actually burned the CPU" — the inclusive form double-counts up the same-thread stack.

`db.result.fetch_size` is not fetch count. It is defined as *"the number of items **requested** in fetching query results"* — a fetch-size hint, not a count of rows returned. It is also `experimental`, which rules it out for alerting. If a classic dashboard tracked fetch count as an N+1-query signal, the durable replacement is span *count* per trace against the database service, not this field. **DBMON-05** (query analysis) covers N+1 detection.

What to do about the gap. There is no reconstruction of IO or lock time from the fields that do exist — `duration` minus `span.timing.cpu` is elapsed-minus-CPU, which lumps IO, lock, queueing, and downstream wait into one undifferentiated number. It can be a useful *triage* signal ("this span is not CPU-bound"), but do not present it as IO time. Where the distinction genuinely drives a decision, the durable path is the child spans themselves: a database call, an outbound HTTP call, and a queue wait each appear as their own span with their own `duration`.

> **Sources:** Distributed Tracing (DT docs), Semantic Dictionary (DT docs). **Dictionary:** `span.timing.cpu` (`stable`), `span.timing.cpu_self` (`stable`); no row for IO time, disk IO time, lock/sync time or suspension under `filter contains(name, "timing") or contains(name, "io.time") or contains(name, "lock") or contains(name, "suspension") or contains(name, "wait")`, which returns exactly six fields — `compilation_timings.*` ×3, the two CPU fields, and `azure.storage.blob.block_id` (`experimental`; an unrelated substring hit on "lock") — so the query surface works and the

> absence is meaningful, not an empty result. Read 08/04/2026, re-verified 08/27/2026; the sixth field appeared by 09/28/2026. **Derived:** the `duration - span.timing.cpu` triage caveat follows from the two published CPU definitions plus the absence of any wait-time field — no source endorses it as a substitute.

Confirm the timing inventory yourself. This searches the entire dictionary — not just spans — so an empty result for lock/wait/suspension is a statement about the platform, not about your tenant's data:

```
// Every timing-related field the semantic dictionary publishes, anywhere.
// Expect: span.timing.cpu, span.timing.cpu_self, compilation_timings.*, and
// azure.storage.blob.block_id (an unrelated "lock" substring hit) — nothing
for IO,
// lock/sync time or suspension.
fetch dt.semantic_dictionary.fields
| filter contains(name, "timing")
    or contains(name, "lock")
    or contains(name, "suspend")
| fields name, stability, type, description
| sort name asc
```

4. Why an External Call Has No Service and No Endpoint

The customer premise here is correct, and it follows from one sentence in the dictionary.

`endpoint.name` is `stable`, and its definition reads: *"The endpoint name is derived from endpoint detection rules and uniquely identifies one endpoint of a particular service… Endpoints are **exclusively detected on request root spans**."*

> **Breaking (SaaS 1.347 — staged rollout from 09/16/2026): endpoint names keep their volatile segments** — for customers using URL truncation, that is Service Detection v2, or Service Detection v1 with enhanced endpoint naming enabled. Verbatim: *"Volatile segments are no longer dropped. Instead, they're replaced with typed placeholders, and path processing continues for the full path."* This does not change *where* endpoints are detected — the root-span rule below still holds — but it changes the **shape of every name**. A path that previously truncated at its first volatile segment now runs its full length with `{id}` / `{token}` standing in for the

> volatile parts, so a filter written against a truncated name stops matching. Cardinality moves with it: two URLs that used to collapse to the same truncated endpoint can now resolve to two names. Re-check endpoint-scoped filters, dashboards and alerts once 1.347 reaches your tenant; until it does, the truncating behaviour is what you will observe.

A client span — the caller's side of an outbound HTTP, gRPC, or database call — made **while serving an inbound request** is not a request root span. So:

On a request root (server) span	On a client span beneath a server span
`endpoint.name` populated	`endpoint.name` **null**
`request.is_failed` populated	`request.is_failed` **null**
Contributes to `dt.service.request.*` series	**No** service/endpoint metric series

The exception: a client span that starts a request. The dictionary defines the request root as the span that *"starts the request within a service"* (`request.is_root_span`). An outbound call from a scheduler, background thread, load generator or batch job has no inbound span above it, so its client span *is* the request root — and carries `endpoint.name` , `request.is_failed` and a `dt.service.request.*` series. On the validation tenant over 2 h (10/15/2026), 17,970 of 478,373 client spans (3.8%) were request roots; every one had both fields, no other client span had either, and the most frequent endpoint among them (`okhttp3.RealCall.execute` , 9,914 spans) had a matching `dt.service.request.count` series.

The last table row is the one with real consequences. The `dt.service.request.*` family — `count` , `failure_count` , `response_time` — is dimensioned by `dt.entity.service` and `endpoint.name` . With no endpoint on a client span made while serving a request, there is no series describing that outbound call to a third-party API. Golden-signal dashboards for external dependencies are built from spans, not from that metric family.

Two consequences worth internalizing:

1. **Characterize dependencies from the client spans.** Group on `server.address` (`stable`). `peer.service` is not in the semantic dictionary; it appears only when OTel instrumentation sets it — treat it as optional, never as the key. **SPANS-04** §3–4 is the full treatment — service dependency mapping, inbound/outbound ratios, slowest-dependency queries.
2. **`request.is_failed` is not your failure signal here — and is deprecated anyway.** The dictionary marks it `deprecated` , describing it as *"considered failed according to*

the failure detection rules. Only present on the request root span." The successor namespace `dt.failure_detection.verdict` / `.results` is `experimental`. For client spans the `stable` answer is `span.status_code` plus the protocol status field (`http.response.status_code`; for gRPC `rpc.grpc.status_code`, which is `experimental`) — which is exactly what **SPANS-03** already teaches. Do not build alerting on the failure-detection namespace while it is experimental.

> **Sources:** What's new in Dynatrace SaaS 1.347 (DT docs) — the endpoint-naming change for volatile segments quoted above, Enhanced endpoints for SDv1 (DT docs) — *"No endpoints are created for external services, background activity services, queue listener services, or key value store services."* Service failure detection (DT docs). **Dictionary:** `endpoint.name` (`stable`), `request.is_failed` (`deprecated`), `dt.failure_detection.verdict` (`experimental`), `span.status_code` (`stable`), `server.address` (`stable`), `rpc.grpc.status_code` (`experimental`), `request.is_root_span` (`deprecated`, read 10/15/2026); quoted definitions read 08/04/2026, re-read 08/27/2026 unchanged; no row for `peer.service`, read 09/28/2026 (0 of 507,133 spans in a 2 h window carried it, against 438,899 with `server.address`). The null-on-client / populated-on-server contrast was observed on live spans on the same tenant and date; the request-root client-span counts and the `dt.service.request.count` series were executed 10/15/2026.

Reproduce the contrast in your own tenant. Two queries, run back to back — the first returns client spans, nearly all with nulls in the endpoint and failure columns (a row that has them is a client span that started its request); the second returns both fields populated:

```
// Client spans: endpoint.name and request.is_failed are null on client spans
beneath a server span;
// a client span that starts a request (no inbound span above it) carries
both.
// server.address is what you group outbound dependencies by.
fetch spans, from:-2h
| filter span.kind == "client"
| fields span.name, server.address, endpoint.name, request.is_failed,
duration, span.timing.cpu
| limit 10
```

```
// Request root (server) spans: both fields are populated.
fetch spans, from:-2h
| filter span.kind == "server" and isNotNull(endpoint.name)
| fields span.name, endpoint.name, request.is_failed, span.timing.cpu,
span.timing.cpu_self
| limit 10
```

5. `request_attribute` vs `captured_attribute`

Both are `stable`. Both are registered in the span model under templated names — `request_attribute.__attribute_name__` and `captured_attribute.__attribute_name__`, where `__attribute_name__` is the name you gave the request attribute in its configuration. So the answer to *"is `request_attribute.<name>` a GA and stable attribute mapping?"* is **yes**.

The useful part is the distinction between the two, because nothing in the name signals it:

	`request_attribute.<name>`	`captured_attribute.<name>`
Stability	`stable`	`stable`
Type	array	array
Scope	Request-scoped, **reconciled**	Span-scoped, **raw**
Definition	*"the request scoped reconciled values of the attribute … defined by the request attribute configuration"*	*"the span scoped raw values that were captured under the name … "*
Reach for it when	You want the attribute's settled value for the request as a whole	You want to see what a specific span actually captured, before reconciliation

Three traps, in the order teams hit them:

1. **The type is array, not scalar.** `filter request_attribute.order_id == "A-1"` does not do what a classic request-attribute filter did. Use `matchesValue()` for membership, or an array function when you need the element. This is the single most common failure and it fails *silently* — an equality comparison against an array simply matches nothing.
2. **Mixed capture types collapse to string.** The dictionary is explicit for `captured_attribute`: if the captured values have mixed types, *"all attributes are*

converted to string and stored as string array." A numeric comparison that worked in one environment can silently stop matching in another where a stray string value appeared.

3. **The name is yours, so the field is not discoverable by browsing.** `request_attribute.__attribute_name__` is a template. Nothing lists your tenant's actual request-attribute fields from the dictionary — you have to read them from the request-attribute configuration, or from a span that carries them.

> **Sources:** Request attributes (DT docs). **Dictionary:** `request_attribute.__attribute_name__` (`stable`) and `captured_attribute.__attribute_name__` (`stable`) — the dictionary names them with the `__attribute_name__` placeholder; definitions, array typing and the mixed-type-to-string rule quoted, read 08/04/2026, re-read 08/27/2026 unchanged. **Derived:** the trap ordering in the numbered list reflects how the array typing interacts with ordinary DQL equality — no source ranks them.

Check both fields' contract before you write the filter:

```
// The two request-attribute field families, with their published contracts.
fetch dt.semantic_dictionary.fields
| filter contains(name, "request_attribute")
    or contains(name, "captured_attribute")
| fields name, stability, type, description
```

6. Answering the Next One Yourself

The three questions above share one method. It generalizes to every *"does Grail have X?"* question, and it costs nothing to run.

The four-case test. Given a field name you expect to exist, `dt.semantic_dictionary.fields` puts it in exactly one of four states:

Result	Meaning	What to do
Row with `stability: stable`	Modeled and supported	Build on it
Row with `stability: experimental`	Modeled, contract may change	Query it; do not alert on it

Result	Meaning	What to do
Row with `stability: deprecated`	Modeled, being removed	Migrate off it
No row	**Not modeled**	Stop looking for it in span data — redesign around what exists

Why this beats querying the data. A query that returns zero rows cannot distinguish "this field does not exist" from "this field exists but is empty in my timeframe" from "I lack the scope to read this table." The dictionary distinguishes them, and it does so without scanning billable bytes. This matters most for exactly the claim this entry makes — *"IO time is not in the model"* — which would be unsafe to assert from an empty span query and is safe to assert from an empty dictionary result.

> **Derived:** the four-case table restates the observed `stability` values plus the absent-row case; no single source presents it as a decision procedure.

The general-purpose lookup. Change the filter and keep the shape:

```
// Does this field exist, and can I build on it?
// Substitute any field-name fragment. An empty result means "not modeled".
fetch dt.semantic_dictionary.fields
| filter contains(name, "timing")
| fields name, stability, type, description
| sort stability asc, name asc
```

```
// Which model owns a field, and what else ships alongside it?
fetch dt.semantic_dictionary.models
| filter data_object == "spans"
| expand fields
| filter contains(fields, "request_attribute")
| fields name, title, fields
```

7. Recommended Approach

If you are porting classic PurePath analysis onto span DQL:

1. **Inventory before you port.** Run the section 2 query once and diff the published field list against the fields your classic dashboards and alerts depend on. The fields with no row are the real work; everything else is a rename.
2. **Replace sub-timings with child spans, not with arithmetic.** Where IO or lock time drove a decision, the durable signal is the child span — a DB call, an outbound HTTP call, a queue wait — each with its own `duration`. `duration - span.timing.cpu` is a triage hint, not a metric to publish.
3. **Use `span.timing.cpu_self` for CPU attribution**, and reserve the inclusive `span.timing.cpu` for whole-subtree questions.
4. **Build external-dependency views from client spans** keyed on `server.address` (`peer.service` only where your OTel instrumentation sets it), and do not expect a `dt.service.request.*` series for calls made while serving a request — only a client span that starts a request gets one. **SPANS-04** has the queries.
5. **Filter request attributes as arrays.** `matchesValue()`, not `==`. Decide deliberately between the request-scoped reconciled value and the span-scoped raw one.
6. **Check `stability` before anything reaches an alert.** `experimental` fields are legitimate for investigation and wrong for paging.

On the DPS question that usually arrives with these — trace Ingest & Process, Retain, and Query are documented in **FINOPS-01** §6, including the footgun that Traces-Ingest meters `ingested_bytes` while Retain and Query meter `billed_bytes`. To find out whether a given platform app consumes trace Query budget in *your* tenant, FINOPS-01 §9 has the pattern: filter `dt.system.events` to `event.type == "Traces - Query"` and group by `client.application_context`. On the validation tenant used for this entry no `Traces - Query` records existed in a 30-day window, so this entry makes no claim about which apps appear — run it yourself rather than assuming either answer.

> **Sources:** Semantic Dictionary (DT docs), Request attributes (DT docs). The zero-`Traces - Query`-records observation is from the validation tenant on 08/04/2026 and is reported as a limit on this entry's evidence, not as a property of the platform.

8. Summary and Next Steps

Five things to carry away:

1. **The span model is published data, not folklore.** `dt.semantic_dictionary.models` and `dt.semantic_dictionary.fields` answer "does X exist and can I trust it" in one query, at zero billable bytes.
2. **CPU time survived; IO, lock, disk IO, and suspension were never modeled.** Absent from the dictionary is a stronger statement than absent from a query result — and it is the statement that justifies calling this permanent rather than pending.
3. **External calls are client spans.** A client span made while serving a request has no `endpoint.name`, no `request.is_failed` and no `dt.service.request.*` series; one that starts a request has all three. Group on `server.address` either way.
4. **`request_attribute.<name>` is `stable` — and is an array, request-scoped and reconciled.** `captured_attribute.<name>` is its span-scoped raw sibling. Equality against either matches nothing.
5. **Check `stability` before you alert.** `request.is_failed` is `deprecated` and `dt.failure_detection.*` is `experimental`; `span.status_code` is `stable`. The failure-detection namespace is not yet somewhere to build.

If you need…	Read
Span DQL fundamentals and the full field walkthrough	SPANS-01, SPANS-02
Failure and error analysis on client spans	SPANS-03
Service dependency mapping from client spans	SPANS-04
Trace Ingest / Retain / Query billing and query-side chargeback	FINOPS-01
Span query cost and the filter-early pattern	SPANS-08
The metrics half of the Classic-vs-Grail question	FAQ-11
Migrating classic entity selectors to Smartscape	FAQ-16

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.